QKDecode: Report

Pritesh Thakur Swechchha Adhikari Anamol Neupane Anju Dhakal
Atal Chalise Bibhuti Ranabhat Chirag Dahal Drone Chaudhary

Live demo: `https://qkdecode-frontend.vercel.app/`

Abstract

Classical key distribution relies on computational hardness assumptions that quantum computers threaten to break, motivating key exchange schemes whose security instead rests on the laws of physics. This report presents QKDecode, a Python-based simulation of the BB84 quantum key distribution protocol, covering the full pipeline from photon preparation and transmission through an optional eavesdropper, detection, sifting, quantum bit error rate (QBER) estimation, error correction, privacy amplification, and XOR-based message encryption. The system is built as modular components (Alice, Bob, the quantum channel, Eve, and the detector) coordinated by a central simulator, with configurable parameters for channel noise, photon loss, detector efficiency, dark counts, and the QBER security threshold. Four experiments were run, each averaged over 20 trials, to characterize how channel noise, photon loss, detector efficiency, and photon count affect QBER, sifted and final key length, and runtime. The results confirm the expected theoretical behavior: QBER rises approximately linearly with channel noise and sharply degrades key security past the 11% threshold, while photon loss and detector efficiency affect the amount of usable key material without measurably changing the error rate, and runtime scales linearly with the number of photons transmitted. These results validate that the simulation correctly reproduces the security guarantees and failure modes of BB84.

1 Introduction

1.1 Classical Cryptography

Cryptography is the practice of hiding a message so that only the intended recipient can read it. In classical cryptography, this is usually done using a **shared secret key**, a string of bits known only to the sender and the receiver. The key is used to scramble the message (encryption) and to unscramble it again on the other end (decryption). Our own implementation uses a simple version of this idea, the XOR cipher. If both sides have the same key, XOR-ing the message with the key hides it, and XOR-ing the ciphertext with the same key reveals the original message again.

The real challenge in cryptography is not the encryption itself, but **how two parties agree on the same secret key in the first place** without anyone else learning it. This is known as the key distribution problem. If Alice and Bob have never met and can only communicate over a public channel, how can they agree on a secret that an eavesdropper (commonly called Eve) cannot also learn simply by listening in? Classical methods solve this using math problems that are very difficult to reverse, such as factoring large numbers. The limitation is that this security only holds because these problems are too slow to solve with current computers, not because they are impossible to solve. Quantum computers are expected to be able to solve these problems much faster in the future, which is part of the motivation behind Quantum Key Distribution.

1.2 Quantum Key Distribution

Quantum Key Distribution (QKD) allows two parties to agree on a shared secret key using the physical properties of quantum particles, rather than the difficulty of a math problem. Instead of being secure because it is hard to compute, QKD is secure because of the laws of physics, specifically the two properties discussed earlier in this report: superposition and the fact that measuring a qubit changes its state.

The main idea works as follows. Each secret bit is encoded onto a qubit using a randomly chosen basis, as described in the Basis section above. If an eavesdropper tries to intercept and measure this qubit, they must guess which basis was used. If they guess incorrectly, the measurement disturbs the qubit. This is an effect with no classical equivalent, since observing a classical bit never changes its value. This disturbance later appears as detectable errors when Alice and Bob compare their results. In short, **QKD does not prevent eavesdropping; it guarantees that eavesdropping leaves evidence**. If no unusual errors appear, Alice and Bob can trust that their key is private. If too many errors appear, they know the channel was compromised and discard the key.

1.3 BB84

BB84 is the first and most well known QKD protocol, proposed by Charles Bennett and Gilles Brassard in 1984, which is where the name comes from. This is the protocol implemented in our simulation. At a high level, BB84 works as follows.

- Alice chooses a random bit and a random basis (rectilinear + or diagonal ×) for each qubit she sends and encodes the bit accordingly.
- Bob has no way of knowing which basis Alice used, so he measures each incoming qubit using a randomly chosen basis of his own.
- Afterward, Alice and Bob publicly share *only their bases*, never the actual bit values, and keep only the results where their bases matched. This step is called **sifting**.
- Since Bob's basis choice is random, roughly half of all qubits survive sifting, which matches the results observed in our own experiments in Section 4.
- Alice and Bob then compare a sample of their sifted bits to calculate the **Quantum Bit Error Rate (QBER)**. A low QBER indicates the channel can be trusted, while a high QBER (above roughly 11 percent) indicates possible eavesdropping, in which case the key is discarded.
- If the key passes this check, it undergoes error correction and privacy amplification, producing a final shared secret key that Alice and Bob can use to encrypt real messages.

This is the same pipeline built and tested in this project: photon preparation and transmission, an optional eavesdropper (Eve), detection, sifting, QBER calculation, and finally encryption, all of which are detailed further in the Implementation section below.

2 Implementation

2.1 System Architecture

We implemented the BB84 protocol as a Python-based simulation. The backend is split into separate modules, each responsible for one part of the protocol. The modules are: `alice.py`, `bob.py`, `channel.py`, `eve.py`, `detector.py`, `qkd.py`, `encryption.py`, and `utils.py`. The main `simulator.py` file connects all of these together and runs the full protocol from start to finish. A separate `config.py` file stores all the adjustable parameters, such as noise level, photon loss probability, detector efficiency, and the QBER threshold, so they can be changed in one place without touching any other file.

The overall flow of the implementation is:

Alice → Quantum Channel → Eve → Detector → Bob → Sifting → QBER → Error Correction → Privacy Amplification → Encryption

2.2 Photon Preparation and Transmission

Alice starts the protocol by generating two random lists: one list of secret bits (each bit is 0 or 1) and one list of random bases (either rectilinear '+' or diagonal '×'). She then encodes each bit as a photon polarization state. The encoding rules are:

- Rectilinear basis (+): bit 0 → Horizontal (H), bit 1 → Vertical (V)
- Diagonal basis (×): bit 0 → Diagonal (D), bit 1 → Anti-diagonal (A)

Each photon is its own object (defined in `photon.py`) that carries a record of its entire journey through the protocol, including its original state, whether it was lost, whether Eve intercepted it, and what Bob eventually measured it as.

The number of photons sent is not fixed. It is calculated based on the length of the message to make sure there are enough key bits left after sifting to encrypt the full message. The formula used is:

$$N = \max(64,\ 3 \times 8 \times n) \tag{1}$$

where n is the number of characters in the message. The factor of 8 converts characters to bits, and the factor of 3 is a safety margin to account for photons lost during transmission and sifting.

Once Alice prepares her photons, the `QuantumChannel` module transmits them to Bob. For each photon, the channel independently rolls two random numbers: one to decide if the photon is lost, and one to decide if the photon is affected by noise. If a photon is marked as lost, it never reaches Bob. If it is affected by noise, its polarization state is flipped to the opposite state in the same basis (H ↔ V, or D ↔ A).

2.3 Eve and the Detector

If Eve is enabled, she intercepts photons between the channel and the detector. Eve tries to secretly read each photon: she picks a random basis, measures the photon to guess its bit, and then sends a new photon (based on her guess) on to Bob. The problem is that when Eve guesses the wrong basis, her measurement changes the photon's state. This change creates errors that Bob can see, which is how Eve gets caught.

The probability that Eve picks the wrong basis is $\frac{1}{2}$, and given the wrong basis, the chance that this introduces an error Bob can see is $\frac{1}{2}$. So the expected QBER when Eve intercepts every photon is:

$$\text{QBER}_{\text{Eve}} = \frac{1}{2} \times \frac{1}{2} = 25\% \tag{2}$$

After Eve, the photons pass through Bob's `Detector`. The detector has a configurable efficiency (default 90%), so some photons that survived the channel and Eve are still missed by the detector. There is also a small dark count rate (default 1%) that simulates the detector sometimes firing by accident, even when no photon actually arrived.

2.4 Bob's Measurement and Key Sifting

Bob generates his own random list of measurement bases independently of Alice. For each photon that arrives, he measures it in his chosen basis. If his basis matches the polarization axis of the photon, he gets the correct bit. If it does not match, he gets a random result.

After all measurements are done, Alice and Bob compare their bases over a public classical channel. Any photon where their bases did not match is thrown away. This step is called *sifting*. On average, about half of all photons survive sifting. The sifted bits on both sides should be identical if there was no noise or eavesdropping, and make up the raw shared key.

The QBER is then calculated over the sifted key:

$$\text{QBER} = \frac{\text{number of mismatched sifted bits}}{\text{total sifted bits}} \tag{3}$$

If the QBER exceeds the threshold of 11%, the protocol is aborted and the key is discarded. This threshold comes from the security proof by Shor and Preskill, which showed that BB84 produces a provably secure key as long as the QBER stays below approximately 11% [1].

2.5 Error Correction, Privacy Amplification, and Encryption

If the QBER is below the threshold, the protocol moves to error correction. Alice and Bob compare their sifted bits, and wherever they don't match, Bob's bit is replaced with Alice's bit. This corrects any remaining errors caused by channel noise or detector dark counts.

After error correction, privacy amplification is applied. The corrected key is put through a hash function (SHA-256), which mixes it up and produces a 256-bit final key. This step is important because if Eve intercepted some photons, she might know know a few bits of the raw key. But once we hash it, those bits become useless to her — she would need to know the entire final key to decode anything, which is impossible to guess.

$$K_{\text{final}} = \text{SHA-256}(K_{\text{corrected}}) \tag{4}$$

The final key is then used to encrypt the message using XOR. Each character of the message is first converted to its 8-bit binary representation. The key is repeated as many times as needed to match the length of the message in bits. The ciphertext is produced by XOR-ing each message bit with the corresponding key bit:

$$C_i = M_i \oplus K_i \tag{5}$$

Decryption works the same way, since XOR is its own inverse:

$$M_i = C_i \oplus K_i \tag{6}$$

The backend returns the plaintext, ciphertext, and decrypted message to the frontend, which displays them in the panel so the user can see the full encryption result.

3 Experiments

3.1 Objectives

After building the simulation, we wanted to actually test it and see how it behaves under different conditions, instead of just trusting that the code works. The goal was to check four things: how noise on the channel affects errors, how losing photons affects the size of the final key, how the detector's efficiency affects the key, and how the number of photons affects how long the simulation takes to run. Each experiment was run 20 times for every setting, and the results were averaged. Running it multiple times instead of just once helped smooth out random variation and gave a more reliable picture of the actual trend.

3.2 Methodology

For every experiment, one parameter was changed at a time while everything else was kept at its default value. This way, any change we saw in the results could be linked back to that one parameter, and not to something else changing at the same time. For each setting, the simulation was run 20 times, and we recorded the QBER, the sifted key length, the final key length, the runtime, and whether the session ended up secure or aborted. These 20 runs were then averaged to get the final number used in the tables and graphs below.

3.3 Experiment 1: Effect of Channel Noise on QBER

In this experiment, the channel noise was increased step by step from 0% to 20%, while everything else was left unchanged. Table 1 and Figure 1 show the results.

Noise (%)	QBER (%)	Runtime (s)	Final Key	Secure
0	0.00	0.0056	254.1	20/20
2	1.92	0.0057	255.6	20/20
5	5.42	0.0061	254.6	20/20
10	9.65	0.0059	216.8	17/20
13	13.28	0.0065	25.6	2/20
15	15.62	0.0058	0	0/20
17	17.26	0.0052	0	0/20
20	20.77	0.0054	0	0/20

Table 1: Effect of channel noise on QBER and key security.

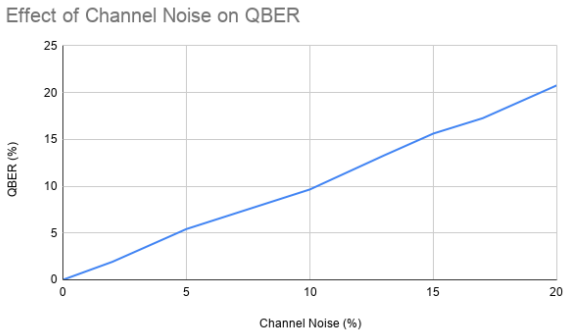

Figure 1: Effect of channel noise on QBER.

As expected, QBER goes up almost in a straight line as noise increases. Below 11% noise, the protocol stays secure almost every time. Once the noise crosses the 11% mark, sessions start getting aborted, and by 15% noise, no session is secure anymore. This matches the theory: since the noise directly causes measurement errors, and the QBER threshold is fixed at 11%, pushing the noise past that point simply makes the channel too unreliable to trust. The runtime stayed roughly the same throughout, which makes sense since noise does not change how many photons are sent, only how many of them end up wrong.

3.4 Experiment 2: Effect of Photon Loss on Sifted Key Length

Here, the photon loss percentage was increased from 0% to 50%, again changing only this one parameter. Table 2 and Figure 2 show the results.

Loss (%)	Sifted	QBER (%)	Runtime (s)	Final Key	Secure
0	445.8	4.90	0.0044	254.8	20/20
5	427.4	5.26	0.0045	254.8	20/20
10	407.4	5.01	0.0045	255.0	20/20
15	385.9	5.44	0.0043	255.2	20/20
20	359.9	4.71	0.0042	255.7	20/20
25	334.1	5.06	0.0043	255.2	20/20
30	316.6	4.59	0.0052	255.0	20/20
35	292.8	4.87	0.0049	255.4	20/20
40	267.2	4.92	0.0045	254.6	20/20
45	248.8	4.64	0.0045	244.7	20/20
50	223.7	5.13	0.0044	223.3	20/20

Table 2: Effect of photon loss on sifted key length.

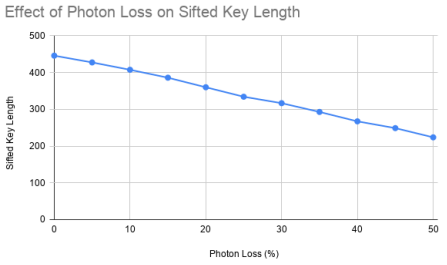

Figure 2: Effect of photon loss on sifted key length.

The sifted key length drops in a steady, almost straight line as photon loss increases, since more lost photons simply means fewer bits reach Bob in the first place. What's interesting is that the QBER barely moves, staying close to 5% the whole time, no matter how much loss there is. This makes sense once you think about it: a lost photon does not become a wrong bit, it just never arrives, so it does not add any extra errors – it only reduces how many usable bits are left over.

3.5 Experiment 3: Effect of Detector Efficiency on Sifted Key Length

In this experiment, detector efficiency was increased from 40% to 100%. Table 3 and Figure 3 show the results.

Efficiency (%)	QBER (%)	Runtime (s)	Sifted	Final Key	Secure
40	5.14	0.0052	181.7	181.7	20/20
50	5.41	0.0053	223.3	223.3	20/20
60	4.75	0.0057	269.6	254.2	20/20
70	4.91	0.0060	314.4	255.3	20/20
80	4.73	0.0061	363.4	255.3	20/20
90	5.10	0.0067	400.9	255.2	20/20
100	4.84	0.0071	447.6	255.2	20/20

Table 3: Effect of detector efficiency on sifted key length.

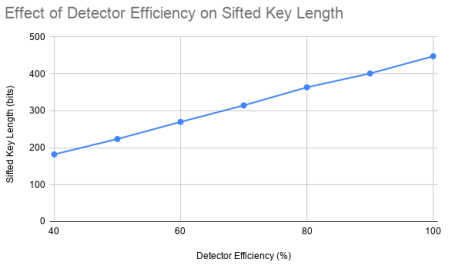

Figure 3: Effect of detector efficiency on sifted key length.

This graph shows the opposite trend from photon loss, but for a related reason. As detector efficiency goes up, more of the photons that actually arrive get detected, so the sifted key grows in a straight line as well. Just like before, the QBER stays flat around 5% regardless of efficiency, since a missed detection just means that bit is gone, not that it turns into an error. Below around 60% efficiency, the final key length also starts falling behind the sifted key length, which shows that very low efficiency can end up limiting how much usable key material is actually available at the end.

3.6 Experiment 4: Effect of Photon Count on Runtime

In this experiment, the number of photons sent was increased from 100 to 10,000 to see how the runtime of the simulation scales. Table 4 and Figure 4 show the results.

Photons	QBER (%)	Runtime (s)	Final Key	Secure
100	5.23	0.0010	33.9	18/20
500	4.57	0.0028	201.8	20/20
1,000	4.71	0.0051	254.6	20/20
2,500	4.93	0.0139	255.1	20/20
5,000	5.12	0.0227	254.7	20/20
7,500	5.11	0.0344	254.7	20/20
10,000	4.99	0.0407	255.2	20/20

Table 4: Runtime scaling with number of photons.

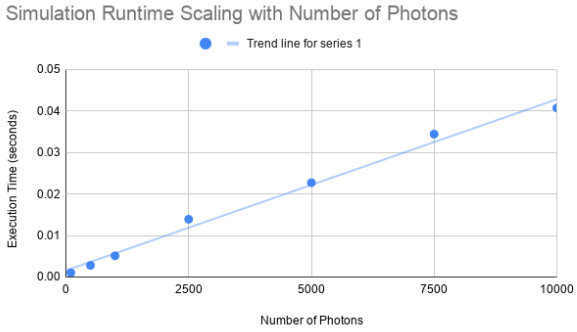

Figure 4: Simulation runtime scaling with number of photons.

The runtime grows in a roughly straight line as the number of photons increases, which makes sense since every photon has to go through the same steps – transmission, detection, sifting, and so on – one after another. At just 100 photons, the key sometimes ends up too short to reliably pass the security check, which is why 2 out of 20 sessions were not marked secure at that setting. Once the photon count reaches a few hundred or more, the results become fully stable. This confirms why the photon count formula in the Implementation section always sends at least 64 photons, and generally a good deal more, so that short messages do not run into this problem.

4 Conclusion

This project implemented and tested a full simulation of the BB84 quantum key distribution protocol, from photon preparation through sifting, QBER-based eavesdropping detection, error correction, privacy amplification, and message encryption. The modular architecture made it possible to isolate and vary individual parameters, such as channel noise, photon loss, detector efficiency, and photon count, and observe their effects independently.

The experiments confirmed that the simulation behaves in line with the theory behind BB84. Channel noise directly drives up the QBER and reliably triggers protocol abortion once it crosses the 11% security threshold predicted by the Shor-Preskill security proof, while photon loss and detector efficiency affect only how much key material survives to the end of the protocol, not its error rate, since a lost or undetected photon simply disappears rather than becoming a wrong bit. Runtime was shown to scale linearly with the number of photons transmitted, and the photon count formula used in the implementation was shown to reliably avoid the short-key failures seen at very low photon counts.

Taken together, these results show that QKDecode correctly reproduces both the security guarantees of BB84, namely that eavesdropping and excessive noise leave detectable evidence in the form of elevated QBER, and its practical trade-offs, namely that physical imperfections such as loss and limited detector efficiency reduce key throughput without compromising security. Future work could extend the simulator to model more realistic error correction protocols such as Cascade, explore other QKD protocols like E91, or evaluate the system's resistance to more sophisticated eavesdropping strategies beyond the simple intercept-resend attack modeled here.

References

[1] P. W. Shor and J. Preskill, *Simple Proof of Security of the BB84 Quantum Key Distribution Protocol,* Physical Review Letters, vol. 85, no. 2, pp. 441–444, 2000. `https://arxiv.org/abs/quant-ph/0003004`